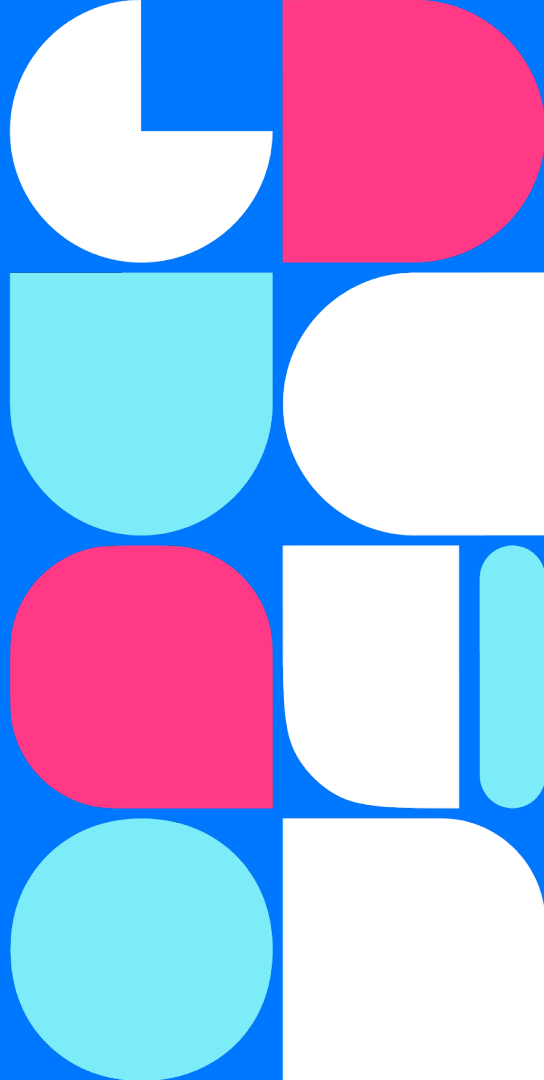
education

Микросервисы

Разработка веб-сервисов на Golang



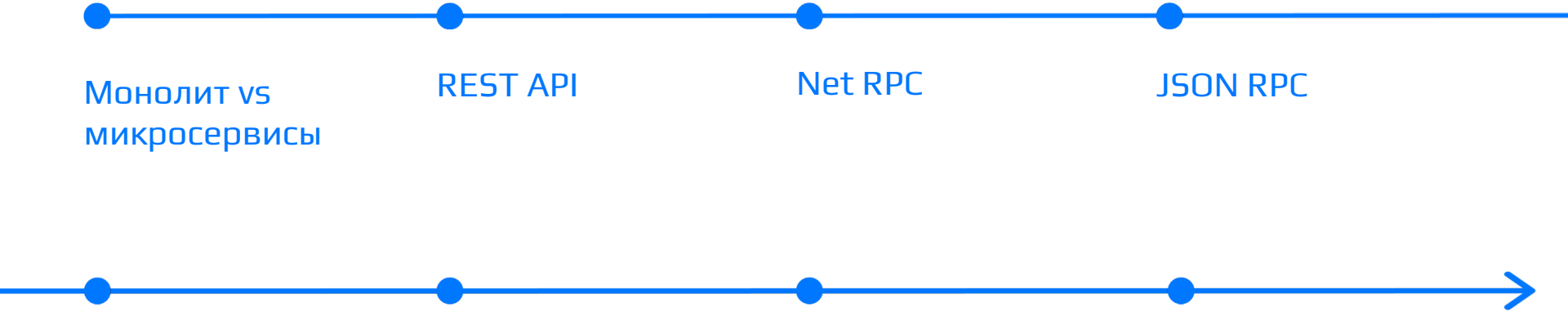
Цель занятия:

1. Обсудить такой популярный паттерн организации проекта, как микросервисы, когда это выгодно, когда нет.

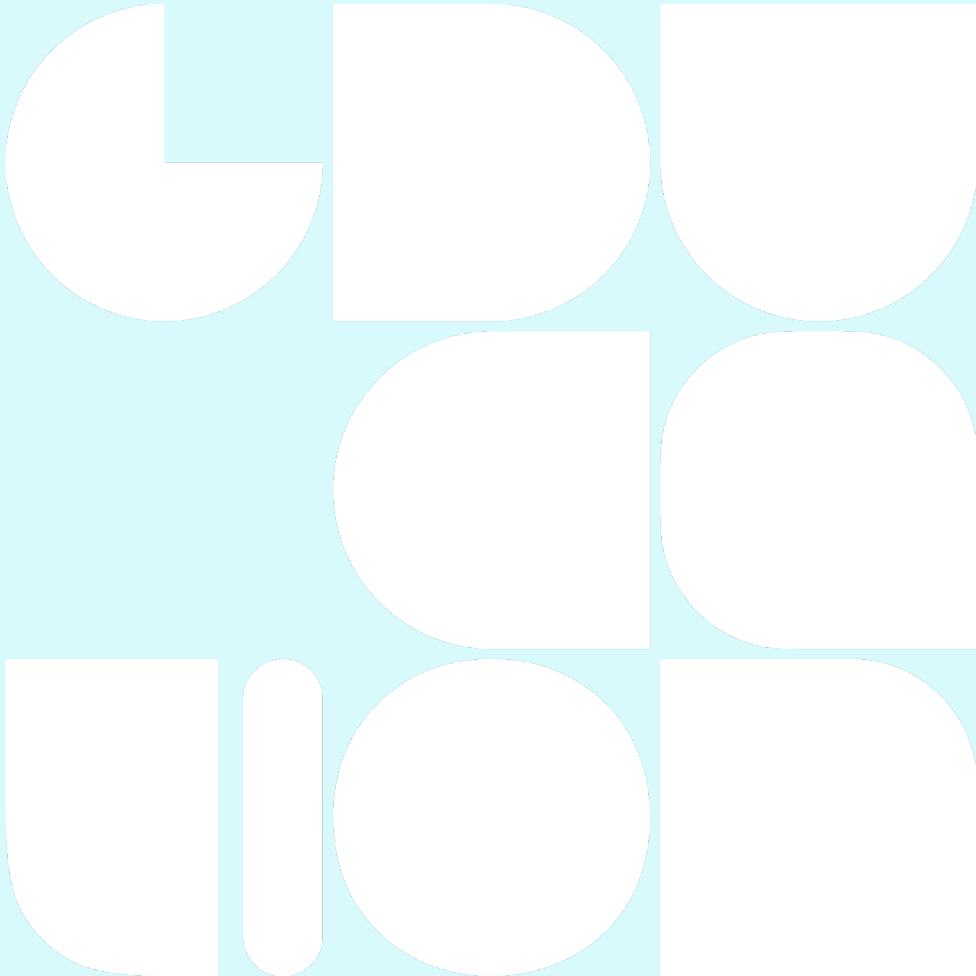
2. Ответить на вопрос, как разрабатывать проект, который в случае чего легко будет распилить на микросервисы.

3. Познакомиться с бинарным форматом сериализации protobuf и gRPC - фреймворком, который упростит вашу жизнь при создании микросервисов.

План лекции

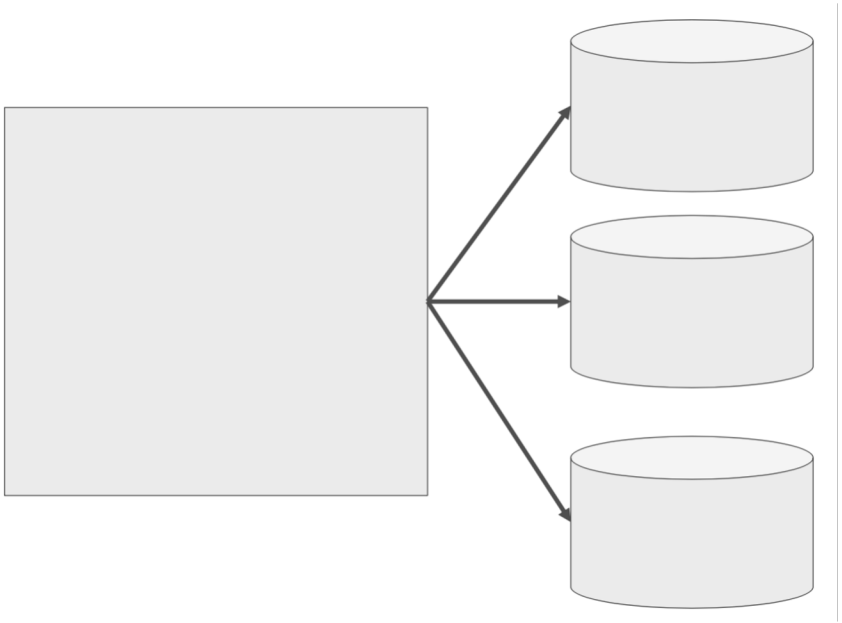

Монолит vs микросервисы
REST API
Net RPC
JSON RPC
Protobuf
gRPC
Пример load balancing и service discovery на Consul
FlatBuffers

Монолит vs микросервисы



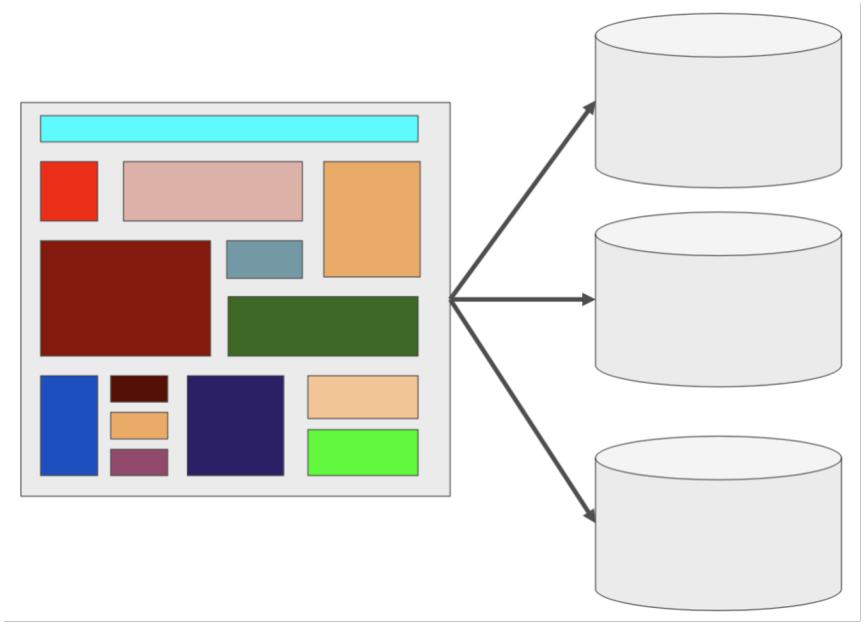
Что такое монолит

- Приложение, которое содержит в себе много очень разнообразной логики
- Ходит во множество баз данных
- Весь код в одном репозитории



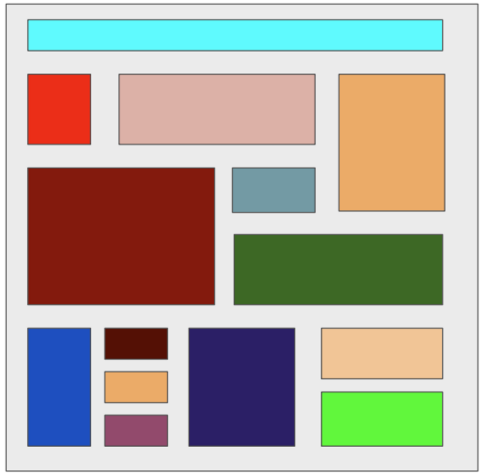
Что такое монолит

- На примере условной соцсети:
  - регистрация
  - авторизация
  - лента
  - видео
  - музыка
  - мессенджер
  - ...



Монолит, сервисы и микросервисы

- А если выделить часть логики в отдельное приложение?
- Сервис - содержит логику определённого функционала веб-приложения (авторизация, отправка писем и т.д.)
- Микросервис - небольшой сервис, который реализует одну конкретную функцию, изолированную от остальной логики веб-приложения (генератор паролей, вычисление хеша и т.д.)



Монолит



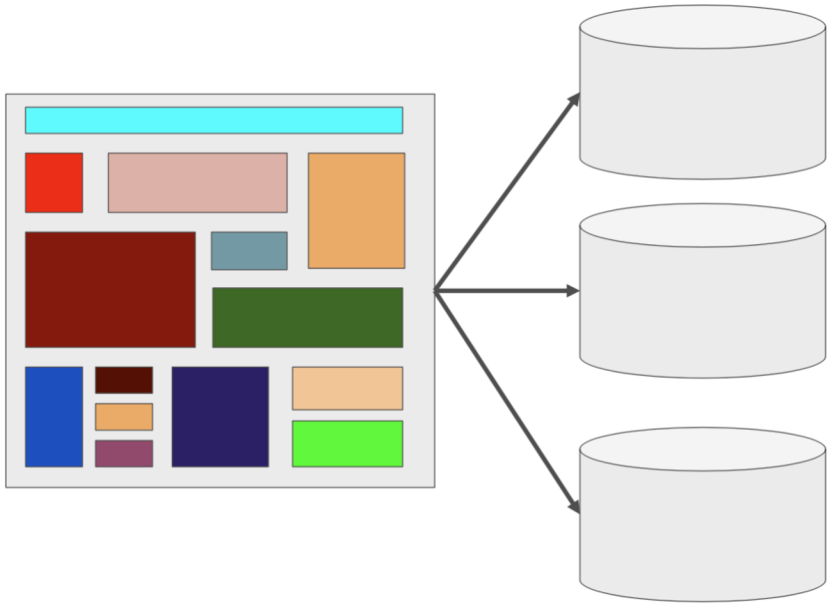
Сервис



Микросервис

Проблемы монолита

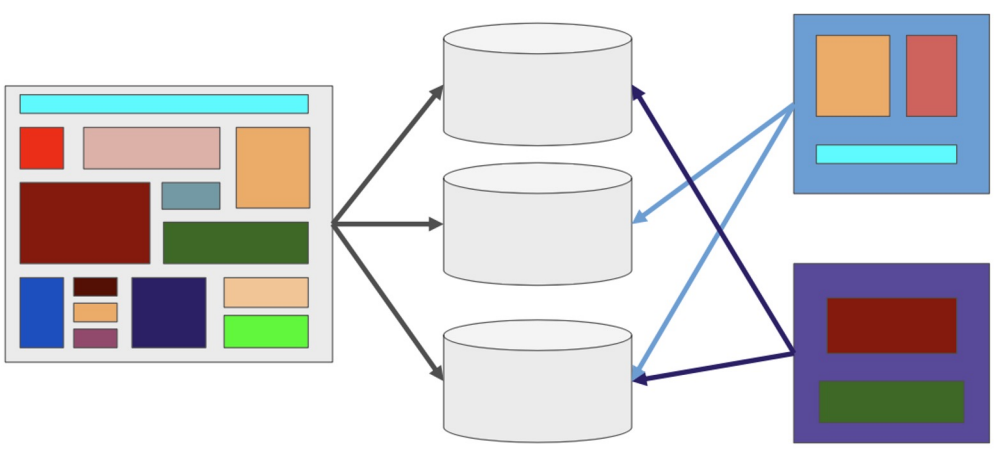
- Большой объём логики, сложно работать с кодом
- Высокая связность, сложно рефакторить и обновлять компоненты
- Много MR’ов в репозитории, постоянные конфликты
- Сложное и долгое тестирование
- Единая точка отказа
- Дублирование кода*



education

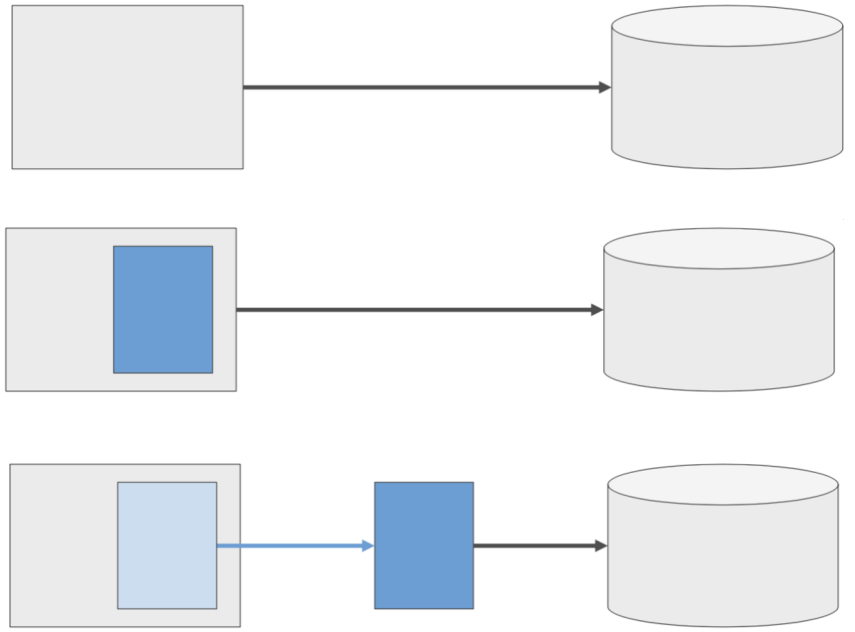
Проблемы монолита. Дублирование кода

- При появлении новых отдельных сервисов вероятно дублирование кода (возможно даже в разных репозиториях)



Разделяем монолит

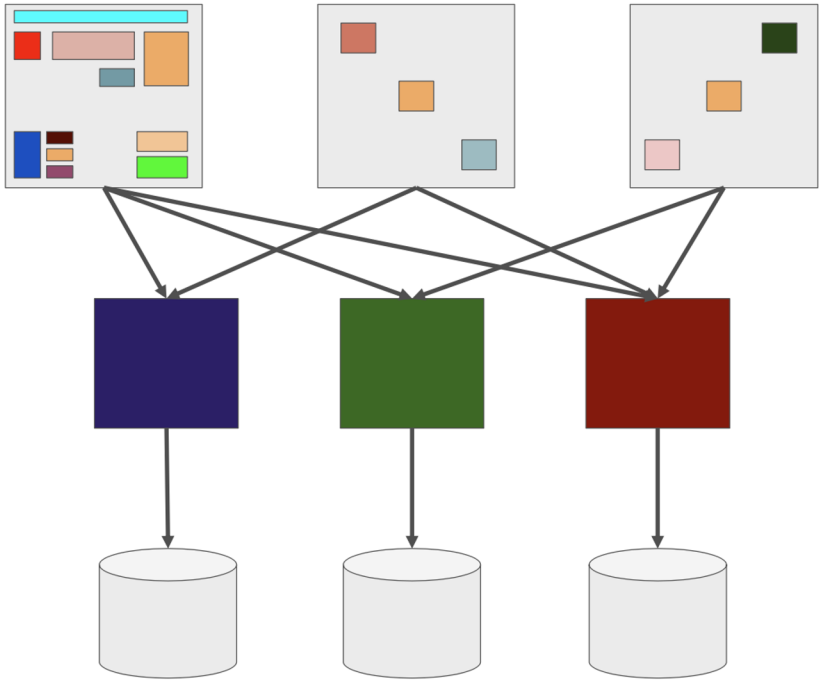
- Возьмём монолит и выделим в нём логику работы с какой-нибудь базой
- Важно уметь декомпозировать код: выделять блоки логики в отдельные пакеты, прятать за интерфейсы и т.д.
- Отдельный блок кода можно выделить в отдельный сервис



education

Нормальная микросервисная архитектура

- Несколько сервисов используют одинаковые микросервисы
- Нет дублирования кода
- Изоляция БД за микросервисом - единая точка доступа к данным
- Но микросервис не обязательно должен ходить в БД
- Легче разработка, тестирование, обновление, масштабирование

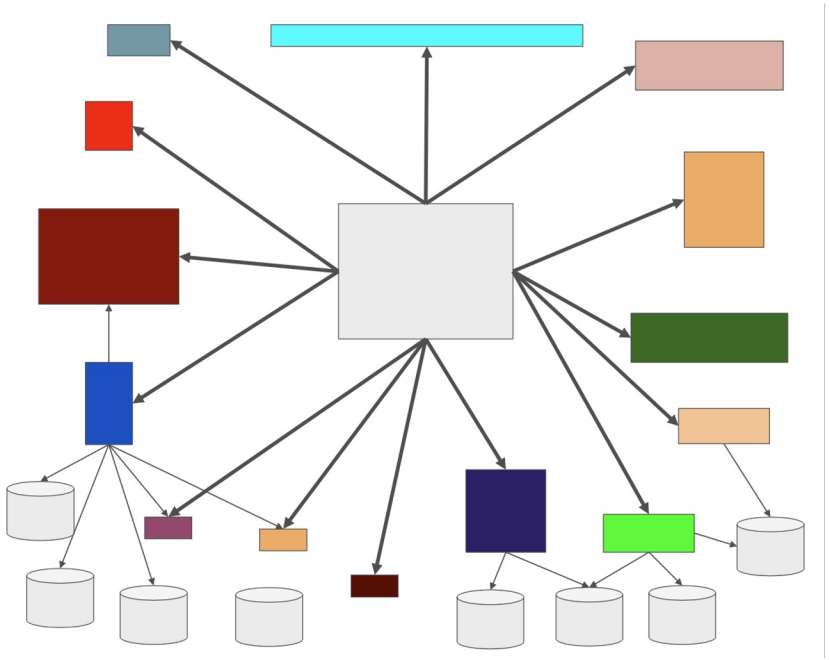


education

Микросервисы. Подводные камни

- Много сетевых походов (а каждый сетевой поход - это долго и ненадёжно)
- Дольше разработка до запуска продукта
- Не подходит для MVP

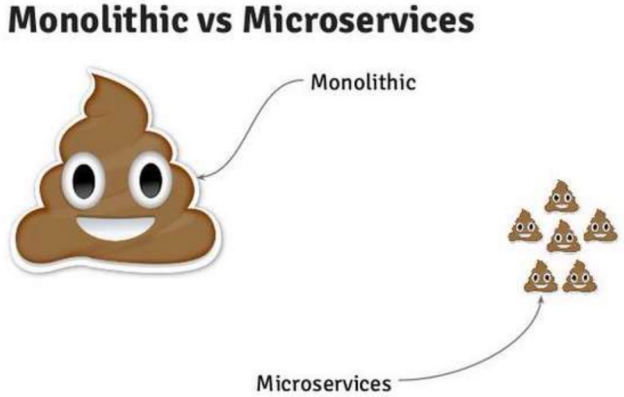
Микросервисы - это в основном про масштабирование ***команд***



education

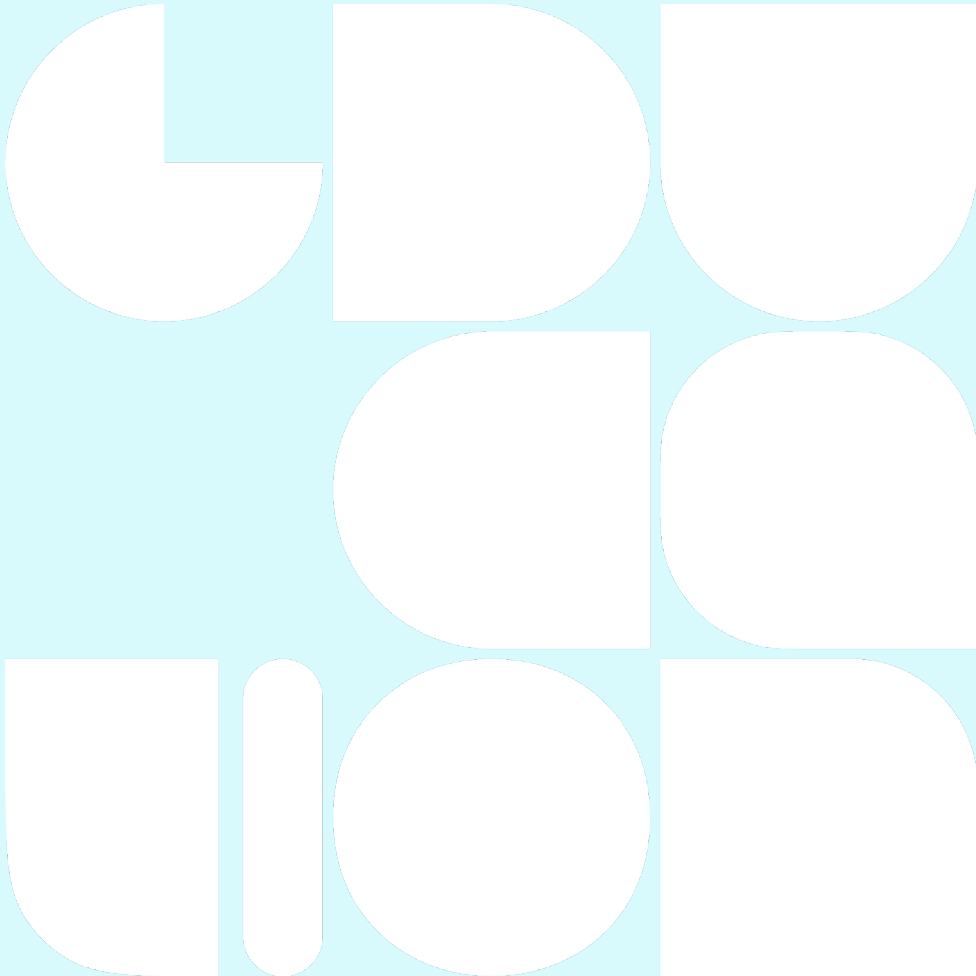
Монолит vs микросервисы

- Микросервисы не решают всех проблем
- Каждая архитектура лучше подходит для разных задач и условий
- Выбирать нужно в зависимости от ситуации
- Классический сценарий: продукт создаётся как монолит и по мере роста распиливается на сервисы/ микросервисы



education

REST API



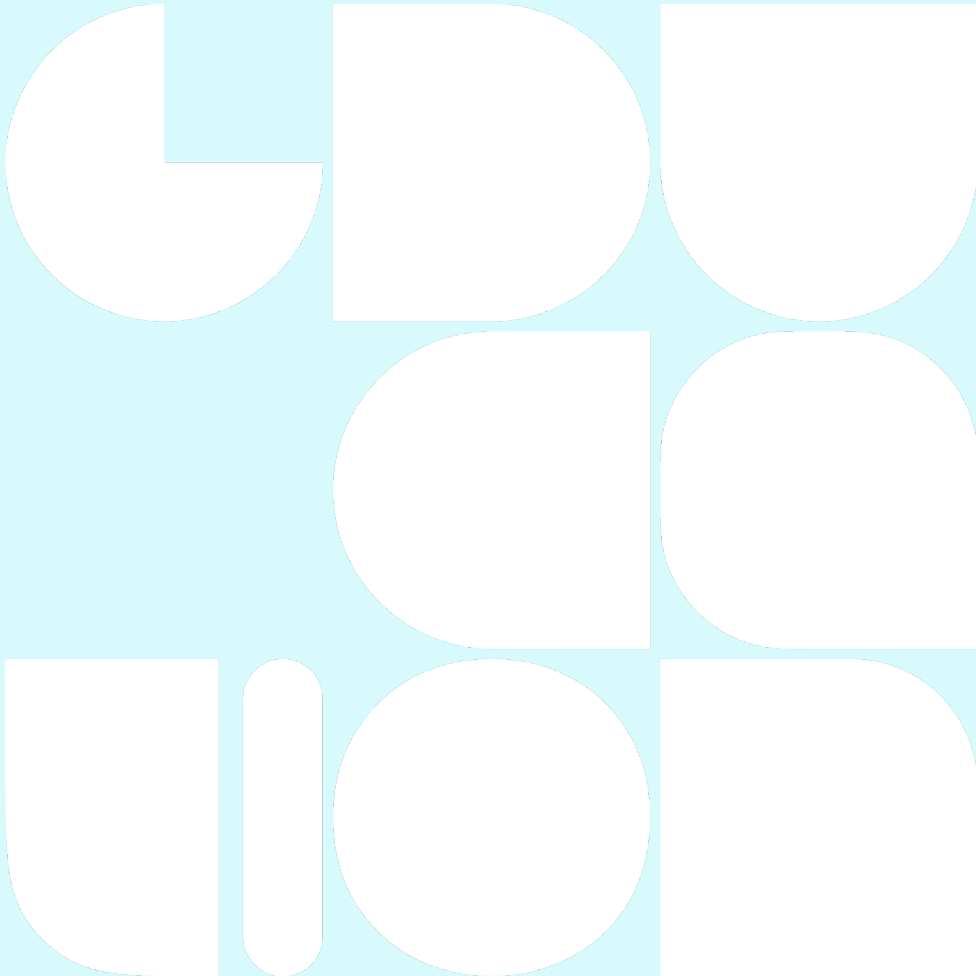
REST API

- REST - Representational State Transfer, «передача репрезентативного состояния»
- Архитектурный подход для создания API на основе взаимодействия по HTTP-протоколу
  - Модель клиент-сервер
  - Отсутствие состояния
  - HTTP-методы, пути, коды ответов...



education

Net RPC



Net RPC

- RPC - remote procedure call, удалённый вызов процедур (или вызов удалённых процедур)
- Подход RPC позволяет вызывать функцию почти напрямую, скрывая логику отправки запроса по сети и обработки ответа

- Пакет net/rpc
  - Для сериализации данных использует бинарный encoding/gob
  - Только для общения между go-программами
  - Заморожен (https://github.com/golang/go/issues/16844)

JSON RPC

JSON RPC

- JSON - JavaScript Object Notation
  - Текстовый -> легко читаемый, но избыточный
- JSON RPC
  - Версия 2.0 - клиент-серверная модель
  - Запрос - JSON-объект с полями jsonrpc, method, id и params
  - Ответ - JSON-объект с полями jsonrpc, error, id и result
  - It is designed to be simple!



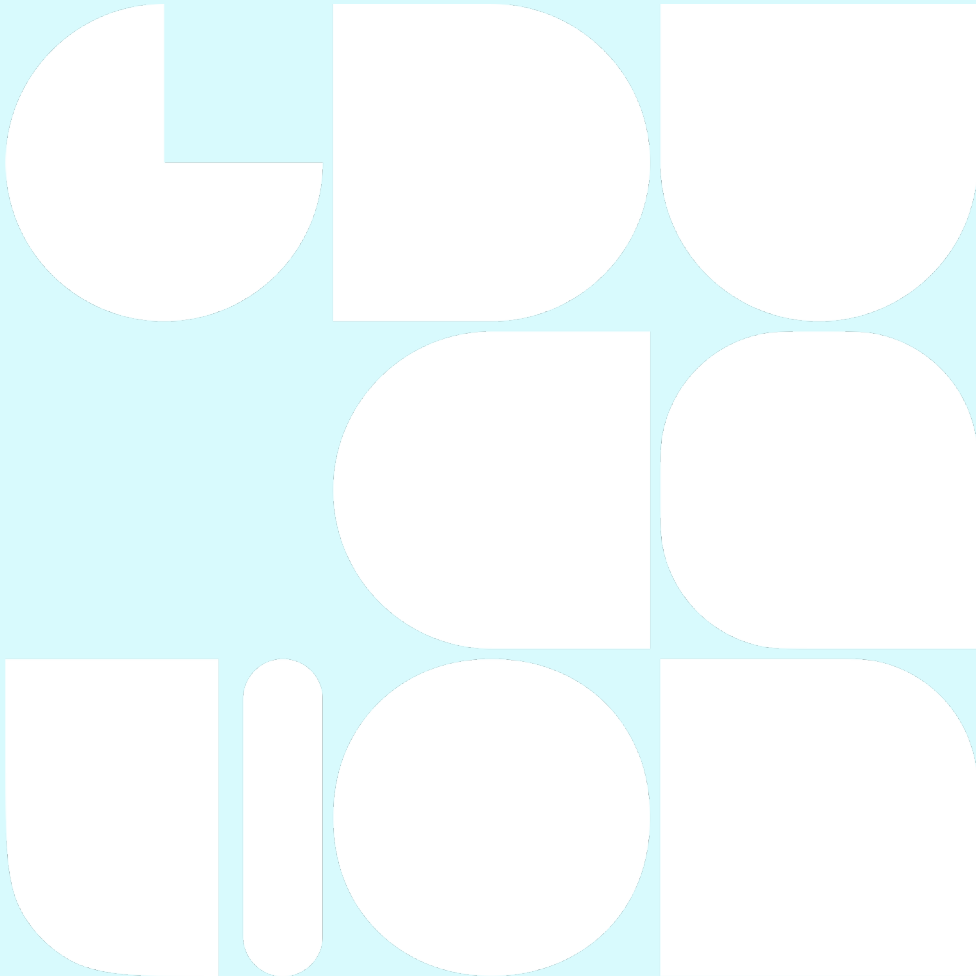
education

Protobuf

Protobuf

- Protobuf - Protocol Buffers, бинарный механизм сериализации данных
- Разработан в Google
- Независим от языка и платформы
- Структура данных описывается в proto-файле
- С помощью компилятора protoc генерируется код сериализации
- Быстрый и эффективный

gRPC



gRPC

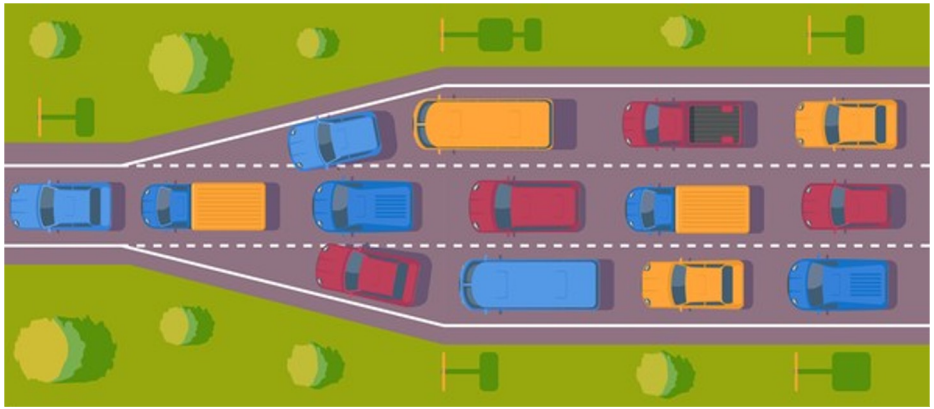
- Высокопроизводительный фреймворк для RPC с открытым кодом
- Разработан в Google
- Независим от языка и платформы
- Использует profobuf для описания интерфейсов и сериализации данных
- Работает поверх HTTP/2
- Поддерживает двунаправленный стриминг
- Поддерживает обратную совместимость

education

Пример load balancing и service discovery на Consul

Балансировка нагрузки

- Микросервисы можно независимо масштабировать, если где-то возникает bottleneck
- Поднимаются несколько инстансов “тормозящего” сервиса



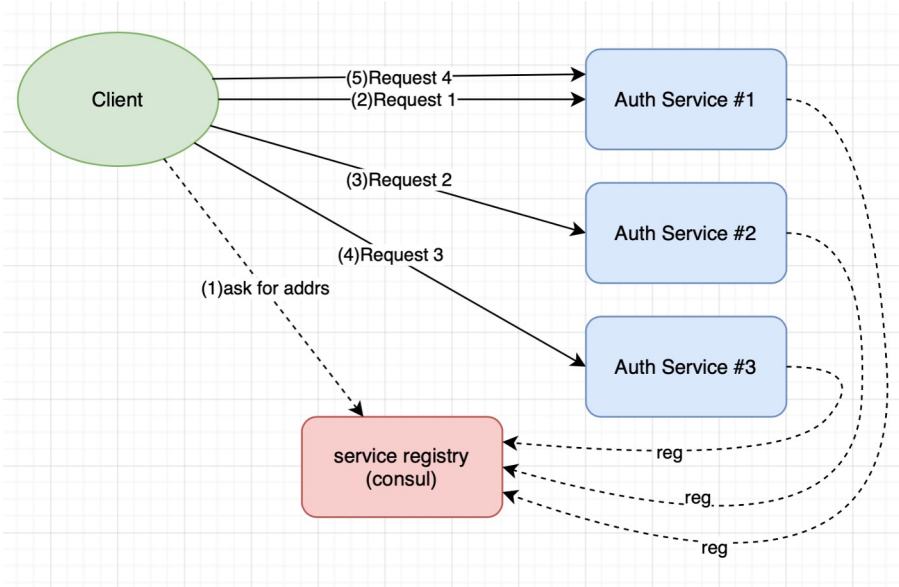
Балансировка нагрузки

- Как одному сервису определить, в какой инстанс другого послать запрос?
  - Как узнать, какие инстансы существуют?
    - нужен механизм обнаружения инстансов
  - Как решить, в какой инстанс отправить запрос?
    - нужен механизм балансировки



Балансировка нагрузки. Consul

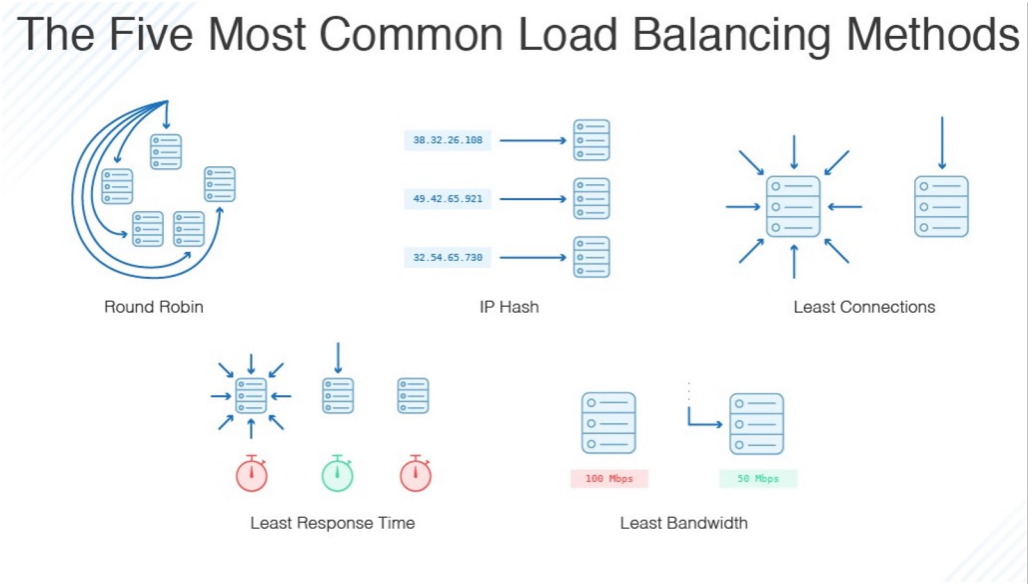
- Consul - один из инструментов с открытым исходным кодом, который используется для обнаружения сервисов (и не только)



education

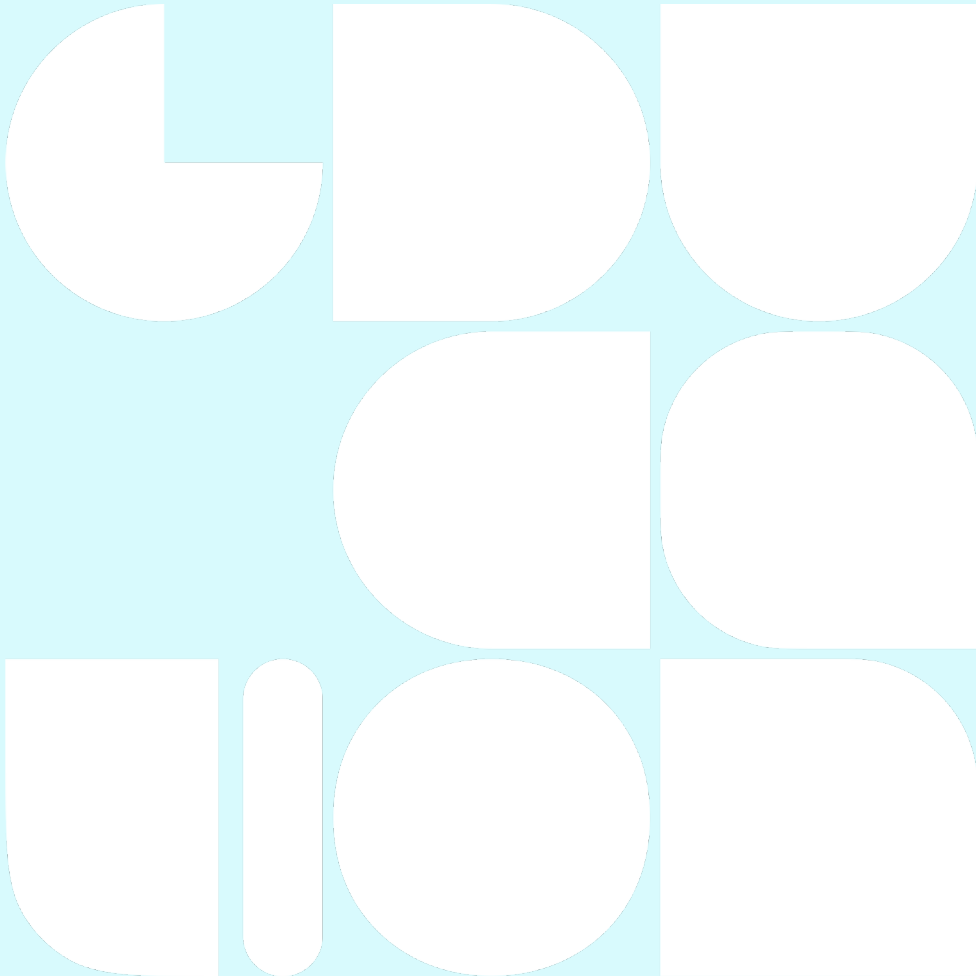
Балансировка нагрузки. Алгоритмы

- Алгоритмов балансировки много и каждый лучше подходит для своей ситуации



education

FlatBuffers



FlatBuffers

- Бинарный механизм сериализации данных
- Разработан в Google
- Позволяет получать доступ к данным без десериализации объекта
- Эффективнее protobuf
- Поддерживает обратную совместимость
- Сложнее в использовании



education

Подведём итоги:

- **Монолит vs микросервисы:**
  - монолит - приложение, которое содержит в себе много очень разнообразной логики, и из-за своей сложности имеет множество проблем
  - микросервисы позволяют решить ряд проблем монолита, но имеют свои недостатки
  - микросервисы - это в основном про масштабирование команд в большом и зрелом продукте

- **REST API**
  - классический, но не очень удобный подход для межсервисной коммуникации по HTTP

- **Net RPC**
  - RPC - технология вызова функции из другого сервиса
  - net/rpc - лёгкий в использовании фреймворк, но не подходящий для продакшена

Подведём итоги:

- **JSON RPC**
  - удобный в использовании и отладке фреймворк, но избыточный при сериализации сообщений
- **Protobuf**
  - быстрый и эффективный механизм сериализации данных
- **gRPC**
  - высокопроизводительный и наиболее популярный фреймворк для межсервисной коммуникации
- **load balancing и service discovery**
  - алгоритмов балансировки много и каждый лучше подходит для своей ситуации, но самым распространённым является Round-robin
- **FlatBuffers**
  - как protobuf, но более эффективный и сложный в использовании

education

Вопросы?



Спасибо
за внимание!

education